Specification-Driven Development (SDD)

A Learner's Guide to Reliable AI-Assisted Engineering

Structured study notes

October 29, 2026

Contents

1 Introduction **3**

2 What Is Specification-Driven Development? **3**

2.1 Definition . . . 3

2.2 A simple example . . . 3

2.3 The SDD cycle . . . 4

3 Why SDD Matters for AI-Assisted Engineering **4**

3.1 The problem with prompt-driven coding alone . . . 4

3.2 What is the source of truth? . . . 4

3.3 A useful analogy . . . 5

3.4 Benefits and limitations . . . 5

4 Specification, Prompt, Code, and Tests **5**

5 Anatomy of a Good Specification **6**

5.1 1. Intent: Why are we building it? . . . 6

5.2 2. Scope: What is included and excluded? . . . 6

5.3 3. Functional requirements . . . 6

5.4 4. Non-functional requirements and constraints . . . 7

5.5 5. Interfaces and contracts . . . 7

5.6 6. Data model and validation . . . 7

5.7 7. Acceptance criteria . . . 8

5.8 8. Test cases . . . 8

5.9 9. Assumptions, dependencies, and open questions . . . 9

5.10 10. Observability and operations . . . 9

6 Keeping the Specification as the Source of Truth **9**

6.1 Use a versioned shared artifact . . . 9
6.2 Use stable requirement IDs . . . 9
6.3 Manage changes deliberately . . . 10
6.4 Traceability . . . 10

7 Using AI Coding Assistants Within SDD **10**
7.1 A practical workflow . . . 10
7.2 Weak prompt versus strong prompt . . . 10
7.3 Review AI-generated tests too . . . 11
7.4 Security and privacy . . . 11

8 Worked Example: Book Management API **11**
8.1 Problem statement . . . 11
8.2 Example requirements . . . 11
8.3 Acceptance criteria . . . 12
8.4 Traceability mapping . . . 12

9 What SDD Is Not **12**

10 Common Failure Modes **13**

11 Reusable Specification Template **13**

12 Practical Exercises **14**

13 Self-assessment questions **15**

14 Glossary **15**

15 Conclusion **16**

1 Introduction

This guide is written for someone learning how to build software with AI assistance. The goal is to understand Specification-Driven Development (SDD), why the specification should be the stable source of truth, how to write a good specification, and how to use it during implementation and testing. The language is direct and practical, with examples relevant to everyday software engineering.

The central principle is: **the specification defines what the software is required to do; a prompt asks an AI tool to perform a particular task.** Prompts may change from one interaction to another, but the agreed specification should remain the reference unless a deliberate change is approved.

We will cover:

1. What SDD is and why it matters for AI-assisted development.
2. Why the specification, not an individual prompt, is the source of truth.
3. The anatomy of a good specification: intent, scope, constraints, interfaces, acceptance criteria, and tests.
4. A practical workflow for using AI coding tools safely and effectively.
5. A worked example, reusable template, common failure modes, and exercises.

2 What Is Specification-Driven Development?

2.1 Definition

Specification-Driven Development is a software engineering approach in which a structured, written specification guides design, implementation, and verification. It describes the intended behaviour, boundaries, constraints, interfaces, and conditions that must be satisfied. Code is developed to meet the specification, and tests provide evidence about whether it does so.

The term SDD is used somewhat differently by different engineering communities. In this guide, it means a practical workflow where the specification is the central maintained reference. It is related to requirements engineering, test-driven development (TDD), behaviour-driven development (BDD), and model-based engineering, but it is not identical to any one of them.

2.2 A simple example

Suppose we want to build a FastAPI service for managing books stored in PostgreSQL. A vague request might be: “Build a book API with create, read, update, and delete functionality.”

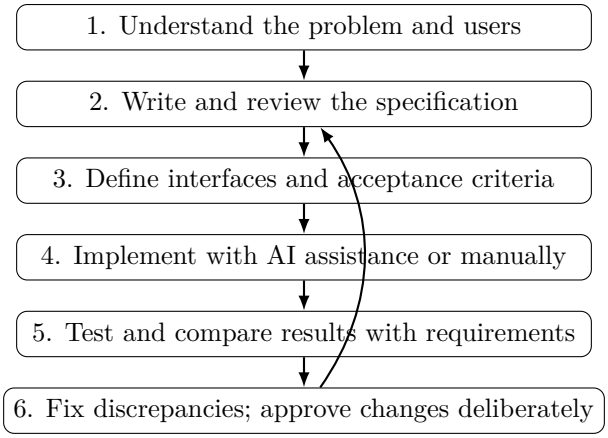
Important questions remain:

- What fields does a book contain?
- Is the ID generated by the server?
- What happens when a book does not exist?
- Which HTTP status codes should each endpoint return?
- Can the price be negative?

- What happens if a required field is missing?
- How should database failures be handled?

A specification resolves these questions. For example, it might state that each book has a generated integer ID, a non-empty title, a category, and a non-negative price; creating a record returns HTTP 201; and retrieving an unknown ID returns HTTP 404. These decisions can be implemented and tested.

2.3 The SDD cycle



This process is iterative. A failed test may reveal a code defect, a missing requirement, or a mistaken assumption. The team should determine which it is rather than silently changing the expected result just to make the test pass.

3 Why SDD Matters for AI-Assisted Engineering

3.1 The problem with prompt-driven coding alone

AI coding tools respond to the instructions and context they receive. If that context is incomplete, the tool may invent assumptions about field names, error handling, dependencies, file structure, or expected behaviour.

For example, separate prompts might ask an AI to create POST, GET, PUT, and DELETE endpoints. If the prompts do not share the same contract, endpoints can become inconsistent: one may return a created object with HTTP 201 while another uses a different response schema or error format. The code may run, but it may not meet one coherent set of requirements.

3.2 What is the source of truth?

A source of truth is the authoritative reference used to determine intended behaviour and decide whether an implementation is correct.

- **Specification:** records agreed requirements and expected behaviour.
- **Prompt:** requests a task from the AI using the specification and relevant context.
- **Code:** implements the requirements.

- **Tests:** check specific requirements under controlled conditions.
- **Review and test results:** provide evidence about whether the implementation meets expectations.

The prompt is not the source of truth because it is often temporary, partial, and task-specific. Today's prompt may request one endpoint; tomorrow's may request a refactor. Neither should silently redefine the API contract. Existing code is not automatically the source of truth either, because it may contain bugs or accidental behaviour.

If code and specification disagree, determine whether the code is wrong or whether a requirement has deliberately changed. Update the specification only when the change is understood and approved.

3.3 A useful analogy

Think of a building project. Approved drawings and specifications describe the intended building. A conversation with a contractor about one room is a task instruction; it does not automatically replace the approved plan. If the plan must change, the change is reviewed and recorded.

In software, the specification is the approved plan, prompts are task instructions, code is the constructed system, and tests are inspections.

3.4 Benefits and limitations

Benefits include consistency across AI sessions, fewer hidden assumptions, testable requirements, traceability, safer refactoring, easier collaboration, and improved maintenance. However, SDD does not guarantee correctness. A specification can be incomplete or wrong, and tests can miss defects. It makes assumptions visible and verification more systematic; it does not remove engineering judgement.

4 Specification, Prompt, Code, and Tests

Artifact	Purpose	Example
Specification	Defines required behaviour and constraints	"A missing book returns HTTP 404."
Prompt	Requests a specific task from an AI tool	"Implement GET /books/{id} according to the API specification."
Code	Implements the behaviour	A FastAPI route that queries PostgreSQL and returns the documented status.
Test case	Checks a requirement	Call GET with an unknown ID and assert status 404.

A good prompt points to the relevant specification, identifies the task and affected files, and states what output is expected. It should not become a competing informal specification.

5 Anatomy of a Good Specification

A good specification need not be extremely long. It should be clear enough that a developer, tester, reviewer, or AI coding assistant can make consistent decisions without guessing about important behaviour.

5.1 1. Intent: Why are we building it?

Intent describes the problem, intended users, and desired outcome. It answers: Who will use the system? What problem are they facing? Why is the current process insufficient? What does success look like?

Example:

> The service will allow an internal library team to maintain a catalogue of books and retrieve book details through a REST API. The goal is to provide consistent, validated access to book records stored in PostgreSQL.

Intent provides context; it is not a substitute for technical requirements.

5.2 2. Scope: What is included and excluded?

Scope defines system boundaries. Write both what is included and what is intentionally excluded.

In scope:

- Create a book record, list books, and retrieve one book by ID.
- Update permitted fields and delete records.
- Validate input and return documented errors.

Out of scope:

- User registration and authentication.
- Reservations, borrowing workflows, or payment processing.
- A graphical frontend.

Explicit exclusions prevent an AI tool from adding features simply because they seem useful. Extra functionality adds maintenance, security, and testing costs.

5.3 3. Functional requirements

Functional requirements describe what the system must do. Give each requirement a stable identifier and make it testable.

- **FR-01:** The API shall create a book when the request contains valid required fields.
- **FR-02:** The API shall return a book by its unique ID.
- **FR-03:** The API shall return HTTP 404 when the requested book does not exist.
- **FR-04:** The API shall reject a negative price.
- **FR-05:** The API shall update only fields permitted by the update contract.

Avoid words such as “fast”, “simple”, or “user-friendly” unless the specification defines how they will be measured.

5.4 4. Non-functional requirements and constraints

Non-functional requirements describe qualities or operating limits. Constraints may arise from technology, security, compliance, or compatibility.

Examples:

- **Performance:** under an agreed test load, 95% of read requests complete within a specified time.
- **Security:** credentials are loaded from environment configuration or an approved secret manager, not hard-coded.
- **Reliability:** database failures produce controlled errors rather than raw tracebacks.
- **Compatibility:** the service uses the agreed Python and FastAPI versions.
- **Maintainability:** routes, validation schemas, and database operations follow the project conventions.

“The API should be fast” is vague. “Under 50 concurrent clients and the agreed dataset, p95 latency for GET /books shall remain below 300 ms” is more testable, if that threshold is realistic.

5.5 5. Interfaces and contracts

Interfaces define how components communicate. For an API, specify routes, methods, request fields, response schemas, status codes, and errors. For a function, specify its name, arguments, return type, exceptions, and side effects.

Method and route	Purpose	Expected behaviour
POST /books	Create a book	Validate input; return created record and HTTP 201.
GET /books	List books	Return list; define pagination and ordering.
GET /books/{id}	Retrieve one	HTTP 200 if found; HTTP 404 if missing.
PUT /books/{id}	Update/replace	Define whether all fields are required and how missing fields behave.
DELETE /books/{id}	Delete a book	Define success response and missing-record behaviour.

The contract must define whether PUT replaces a resource or supports partial updates; many APIs use PATCH for partial updates. Do not leave this ambiguous.

5.6 6. Data model and validation

Define entities, field types, required fields, allowed values, defaults, uniqueness, and validation behaviour.

Field	Type	Rule	Notes
id	Integer	Generated by server	Unique identifier.
title	String	Required, non-empty	Define whitespace handling.
category	String	Required	Enumerate values if needed.
price	Decimal	Must be ≥ 0	Use a suitable decimal representation for money.

A real catalogue may need author, ISBN, currency, timestamps, or inventory count. Include fields because the problem requires them, not merely because they are possible.

5.7 7. Acceptance criteria

Acceptance criteria define the conditions that must be met for a feature to be accepted. They should be observable and testable. A common format is Given–When–Then.

- **AC-01:** Given valid title, category, and non-negative price, when POST /books is submitted, then the API returns HTTP 201 and the created book.
- **AC-02:** Given a negative price, when POST /books is submitted, then the API returns the documented validation error and creates no record.
- **AC-03:** Given a missing required title, when POST /books is submitted, then the API returns the documented validation response and creates no record.

5.8 8. Test cases

Acceptance criteria state what must be true; test cases specify concrete checks. Include normal behaviour, boundary conditions, invalid input, and failure cases.

ID	Scenario	Setup/input	Expected result
TC-01	Create valid book	Title/category; price 100	HTTP 201; generated ID and correct fields.
TC-02	Negative price	Price -1	Validation error; no record created.
TC-03	Missing title	Required title omitted	Documented validation response.
TC-04	Read existing book	ID exists	HTTP 200 and matching record.
TC-05	Read missing book	Unknown ID	HTTP 404.
TC-06	Delete existing book	ID exists	Documented success; record no longer exists.
TC-07	Database unavailable	Connection fails	Controlled error; no credentials or stack trace exposed.

A deterministic test should produce the same expected outcome under the same controlled conditions. Avoid relying on record order unless ordering is specified.

5.9 9. Assumptions, dependencies, and open questions

Write down assumptions instead of allowing the AI to guess. For example:

- PostgreSQL is the approved database.
- Schema changes are managed through migrations.
- Authentication is handled by another service and is outside the current scope.
- The service runs in a defined Python environment.

Unresolved questions should be listed. For example, if the team has not decided whether deleting a missing book returns 404 or is treated as a successful idempotent operation, that decision should be made explicitly.

5.10 10. Observability and operations

For a service that will run in production, define what should be logged and measured: request counts, latency, error rates, database failures, correlation IDs, health checks, and deployment configuration. Specify that passwords, secrets, and sensitive personal data must not be unnecessarily logged.

6 Keeping the Specification as the Source of Truth

6.1 Use a versioned shared artifact

Keep the specification in a stable location, preferably in version control. A Markdown file, YAML/JSON file, OpenAPI document, or linked requirements can work. The important point is that the team knows which version is authoritative.

A simple layout might be:

```
project/
  specs/
    product-requirements.md
    api-contract.yaml
    acceptance-criteria.md
  app/
    main.py
    schemas.py
    database.py
  tests/
    test_books_api.py
  README.md
```

Small projects may use one file; larger projects may separate product, API, security, and operations specifications.

6.2 Use stable requirement IDs

Use IDs such as FR-01, NFR-02, API-03, and TC-05. Stable IDs make it easier to connect code changes, tests, bugs, and reviews to requirements. Avoid changing IDs casually if other artifacts reference them.

6.3 Manage changes deliberately

Specifications should evolve as understanding improves. SDD does not mean the first specification can never change. It means changes are explicit and reviewed.

1. Describe the proposed change and why it is needed.
2. Identify affected interfaces, data, tests, security controls, and documentation.
3. Review and approve the change with the appropriate owner.
4. Update the specification and relevant tests.
5. Implement the change and verify it against the updated specification.

Do not change a test's expected value merely because the current implementation fails. First determine whether the code is defective or the requirement has changed.

6.4 Traceability

Traceability means connecting a requirement to implementation and verification:

$$\text{FR-03} \rightarrow \text{GET /books/\{id\}} \rightarrow \text{test_read_missing_book.}$$

This makes it easier to check whether important requirements have implementation coverage and relevant tests.

7 Using AI Coding Assistants Within SDD

7.1 A practical workflow

1. **Read the specification first.** Confirm the task and constraints.
2. **Ask for a plan.** Have the AI list affected files, requirements, assumptions, and tests before editing.
3. **Resolve ambiguity.** Do not let it invent consequential requirements without confirmation.
4. **Implement a small unit of work.** For example, one endpoint or module at a time.
5. **Review the diff.** Check that changes are necessary and consistent with the spec.
6. **Run tests.** Use unit, integration, contract, and security tests as appropriate.
7. **Update traceability.** Link implementation and tests to requirement IDs.
8. **Record approved changes.** If implementation reveals a missing requirement, update the spec deliberately.

7.2 Weak prompt versus strong prompt

A weak prompt is:

> Make my FastAPI project better. Fix the endpoints and improve the code.

It has no clear success condition. The AI might rewrite working components, change endpoint behaviour, add dependencies, or remove features.

A stronger prompt is:

> Implement requirement FR-03 from `specs/product-requirements.md`. Update only GET /books/{book_id} and its directly related tests. Follow the API contract for response fields and status codes. If the specification is ambiguous, report the ambiguity before changing behaviour. Do not modify the database schema or add dependencies without approval. Run relevant tests and summarise changed files, results, and remaining gaps.

The stronger prompt directs the AI to use the source of truth. It does not replace the specification.

7.3 Review AI-generated tests too

An AI can generate a test that repeats the same mistaken assumption as its implementation. If it incorrectly assumes a missing record should return HTTP 200, it may generate both the endpoint and test around that mistake. Tests are useful evidence only when expected outcomes are grounded in approved requirements.

Review acceptance criteria independently and verify that tests genuinely cover them. Contract checks, boundary tests, code review, and mutation testing can help expose weak tests.

7.4 Security and privacy

Do not send secrets, production credentials, confidential records, or sensitive source code to an AI service unless the organisation has explicitly approved that use. Review AI-generated code for SQL injection, insecure access control, unsafe deserialisation, leaked secrets, vulnerable dependencies, and error messages that reveal internals. Tool permissions must be enforced by application code, not just by a model's promise to follow instructions.

8 Worked Example: Book Management API

8.1 Problem statement

Build a small REST API for maintaining books in PostgreSQL. The scope includes CRUD operations and input validation. Authentication, reservations, payments, and a frontend are excluded.

8.2 Example requirements

- **FR-01:** POST /books creates a valid record and returns HTTP 201.
- **FR-02:** GET /books returns a list in a documented order.
- **FR-03:** GET /books/{id} returns HTTP 200 for an existing record and HTTP 404 for a missing one.
- **FR-04:** Create and update operations reject negative prices.
- **FR-05:** Update behaviour is explicitly defined as full replacement or partial update.

- **FR-06:** DELETE follows documented success and missing-record behaviour.
- **NFR-01:** Database credentials are not hard-coded.
- **NFR-02:** Internal stack traces and secrets are not returned to clients.

8.3 Acceptance criteria

AC-01: Valid creation

> Given the database is available and the request contains a non-empty title, category, and non-negative price, when POST /books is submitted, then the API returns HTTP 201 and the created record with a generated ID.

AC-02: Invalid price

> Given the request contains a price below zero, when POST /books is submitted, then the API returns the documented validation response and no record is created.

AC-03: Missing record

> Given no record exists for the requested ID, when GET /books/{id} is called, then the API returns HTTP 404 with the documented error format.

8.4 Traceability mapping

Requirement	Implementation area	Verification
FR-01	POST route, request schema, database insert	Valid creation and invalid-input tests.
FR-03	GET-by-ID route and repository query	Existing ID returns 200; missing ID returns 404.
FR-04	Request validation	Negative price rejected; database unchanged.
NFR-01	Configuration and connection module	Code review or secret scan.
NFR-02	Exception handling	Simulated database error does not expose internals.

The important point is the link between intent, requirements, acceptance criteria, implementation, and tests—not the book API itself.

9 What SDD Is Not

- **Not a guarantee of correctness:** an incomplete specification can produce a consistently wrong system.
- **Not a ban on prompts:** prompts are useful, but should respect the specification.
- **Not necessarily a massive document:** detail should match project complexity and risk.

- **Not identical to TDD:** TDD organises development around writing tests in short cycles; SDD makes an explicit specification the central reference. They can be used together.
- **Not a reason to ignore judgement:** engineers must challenge contradictions, security risks, and impractical requirements.
- **Not a one-time activity:** update the specification through a deliberate change process.

10 Common Failure Modes

Failure mode	Better practice
Vague requirements	Use observable, measurable statements and examples.
Scope expansion	Document exclusions and approve additions deliberately.
AI silently guesses	Ask it to list ambiguities and pause on important decisions.
Tests copy implementation assumptions	Review expected outcomes against the specification independently.
Specification becomes stale	Version it and update it with approved changes.
No traceability	Link requirement IDs to code and tests.
Only happy paths are tested	Test boundaries, invalid input, missing records, and dependency failures.
Security considered late	Include security and privacy constraints from the start.
Passing tests treated as proof of perfection	Combine tests with review, monitoring, and risk-based evaluation.

11 Reusable Specification Template

Copy this into a Markdown file and adapt it:

```
# Project Specification

## 1. Intent
- Problem being solved:
- Intended users:
- Desired outcome:

## 2. Scope
### In scope
- ...
### Out of scope
- ...

## 3. Functional requirements
- FR-01:
- FR-02:

## 4. Non-functional requirements and constraints
- Performance:
- Security and privacy:
- Reliability:
- Compatibility:
- Operations:
```

```
## 5. Data model and validation
- Entities and fields:
- Types and required fields:
- Allowed values and boundaries:
- Defaults and uniqueness:

## 6. Interfaces and contracts
- Endpoint/function:
- Request:
- Response:
- Status codes and errors:
- Dependencies:

## 7. Acceptance criteria
- AC-01 (Given / When / Then):
- AC-02 (Given / When / Then):

## 8. Test cases
- TC-01: scenario, setup, input, expected result
- TC-02: scenario, setup, input, expected result

## 9. Assumptions and open questions
- Assumptions:
- Unresolved decisions:

## 10. Observability and deployment
- Logs and metrics:
- Health checks:
- Configuration and secrets:
- Deployment environment:

## 11. Change history
- Version, date, change, approval:
```

This is a starting point, not a mandatory checklist for every small script. Use the sections that improve clarity and verification.

12 Practical Exercises

1. Take "Build a book API" and list at least ten missing decisions.
2. Convert five vague requests into uniquely identified, testable functional requirements.
3. Define one endpoint's request fields, response fields, status codes, and error behaviour.
4. Write Given–When–Then criteria for valid input, invalid input, missing records, and database failure.
5. Link each requirement to an implementation module and one or more tests.
6. Give an AI coding assistant a specification and ask it to implement one requirement. Check whether it changed unrelated files or invented behaviour.
7. Change one approved requirement and identify which code, tests, interfaces, and documentation must be updated.

13 Self-assessment questions

1. What is Specification-Driven Development?
2. Why is a prompt not a stable source of truth?
3. How are a specification, code, and tests different?
4. What belongs in intent and scope?
5. What is the difference between functional and non-functional requirements?
6. What information belongs in an API contract?
7. How do acceptance criteria differ from test cases?
8. Why should assumptions and open questions be recorded?
9. How do requirement IDs improve traceability?
10. What should happen when code and specification disagree?
11. How can AI be used without allowing it to invent requirements?
12. Why can AI-generated tests repeat an implementation's mistake?
13. How can SDD and TDD work together?
14. Which security and privacy constraints should be specified?
15. Why does passing a test suite not prove complete correctness?

14 Glossary

Acceptance criteria
Observable conditions required for a feature to be accepted.

API contract
Agreed endpoints, request/response schemas, status codes, and error behaviour.

Functional requirement
A statement describing behaviour the system must provide.

Non-functional requirement
A quality or operating constraint, such as performance, security, or reliability.

Prompt
An instruction supplied to an AI system for a particular task.

Requirement ID
A stable identifier used to reference a requirement.

Source of truth
The authoritative reference used to resolve what is intended.

Specification
A structured description of behaviour, constraints, interfaces, and verification conditions.

Traceability
Connecting requirements to implementation, tests, and change history.

Test case
A setup, input, action, and expected outcome used to verify behaviour.

SDD
Specification-Driven Development; using an explicit specification to guide engineering and verification.

TDD
Test-Driven Development; developing in short cycles organised around tests.

15 Conclusion

Specification-Driven Development provides a disciplined way to build software, especially when AI coding assistants are involved. The specification records agreed intent, scope, constraints, interfaces, and expected behaviour. Prompts direct individual tasks, code implements requirements, and tests provide evidence about whether the result meets them.

The key habit is to avoid letting temporary prompts or accidental code behaviour silently redefine the product. Keep the specification versioned, make changes deliberate, connect requirements to tests, and review AI-generated work like any other engineering contribution. SDD does not remove uncertainty, but it makes decisions visible and gives the team a stronger basis for building and maintaining reliable software.

References

[1] ISO/IEC/IEEE 29148, *Systems and Software Engineering—Requirements Engineering*. https://www.iso.org/standard/72089.html

[2] OpenAPI Initiative, *OpenAPI Specification*. https://spec.openapis.org/oas/latest.html

[3] Cucumber, *Behaviour-Driven Development*. https://cucumber.io/docs/bdd/

[4] Martin Fowler, *Test-Driven Development*. https://martinfowler.com/bliki/TestDrivenDevelopment.html

[5] OWASP, *Application Security Verification Standard*. https://owasp.org/www-project-application-security-verification-standard/